Event Hub(Apache Kafka)

Saturday, 21 February 2026 10:57 PM

Azure Event Hubs: A Managed Event Streaming Service

Introduction

Azure Event Hubs is a fully managed, real-time data ingestion service provided by Microsoft Azure. It is designed to handle massive streams of events from diverse sources, acting as a highly scalable event streaming platform. Event Hubs operates on the fundamental principle of **publish-subscribe (pub-sub)**, also known as the **producer-consumer model**.

The Producer-Consumer Model

In the context of Event Hubs, there are two primary roles:

- **Producer**: An entity that sends events to an Event Hub. Producers can be any application, device, or service that generates data.
- **Consumer**: An entity that reads and processes events from an Event Hub. Consumers can be various analytics services, storage solutions, or other applications.

Example: Taxi Booking System

Consider an organization operating a web application for a taxi booking system, similar to Uber. In this scenario:

1. The **web application** acts as a **producer**. When a user interacts with the application, for instance, by hitting the "book" button, an **event** is created.
2. This event, containing information about the booking, is then sent to Azure Event Hubs. Event Hubs acts like a channel, storing these events in an ordered sequence.
3. For every booking action, a new event is generated and stored in chronological order within the Event Hub.

Event Storage and Consumption

Azure Event Hubs provides temporary storage for these streams of events. Once events are stored, various consumers can access and process them.

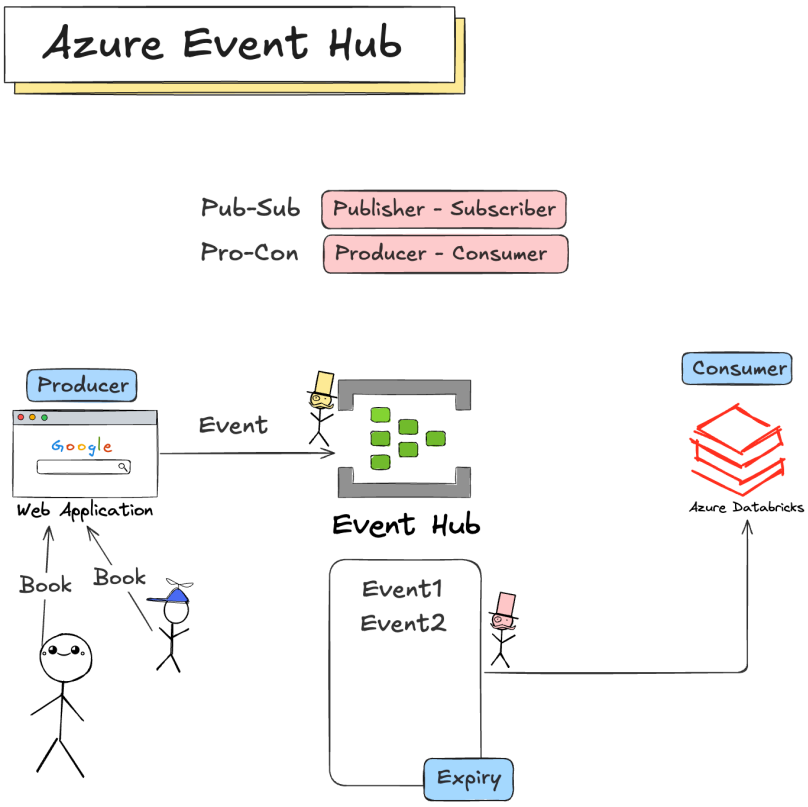
- **Consumers**: Examples of consumers include:
  - **Azure Databricks**: A powerful analytics platform for processing large datasets.
  - **Azure Synapse Analytics**: An integrated analytics service for data warehousing and Big Data analytics.
  - Other custom applications or services that need to react to real-time events.

Event Retention and Expiry

Events in Azure Event Hubs are not stored indefinitely. They have a configurable **expiry date** or retention period. This means that events are only available for a limited time. Before an event expires, consumers must pull the data and process it. This mechanism ensures that storage is managed efficiently and only relevant, recent data is retained for real-time processing.

Comparison with Apache Kafka

Azure Event Hubs is often described as a cloud-native, fully managed **alternative to Apache Kafka**. While both platforms serve similar purposes as distributed streaming platforms, Event Hubs offers the advantage of being a platform-as-a-service (PaaS) offering from Azure. This means that Microsoft manages the underlying infrastructure, scaling, and maintenance, allowing users to focus solely on their data streaming applications without the operational overhead associated with self-managing Apache Kafka clusters.



Policies

Tuesday, 3 March 2026 9:40 PM

Azure Event Hubs: Consumers and Security Policies

Introduction

Azure Event Hubs is a fully managed, real-time data ingestion service that operates on a producer-consumer model. This document outlines how different services act as consumers for Event Hubs and the security policies used to manage access.

Consumers for Azure Event Hubs

A **consumer** is any application or service that reads and processes events from an Event Hub. Consumers are a critical part of a real-time data pipeline, taking raw event data and transforming it into meaningful insights.

Azure Databricks as a Consumer

Azure Databricks is a powerful analytics platform that acts as a consumer by using **Spark Structured Streaming**. * **Real-time Processing**: Databricks establishes a connection to the Event Hub and treats the stream of incoming events as an unbounded table that is continuously updated. * **Scalability**: This allows for real-time data processing and analytics at a massive scale, leveraging the distributed computing power of Apache Spark.

Azure Synapse Analytics as a Consumer

Azure Synapse Analytics is an integrated analytics service that can consume data in several ways: * **Synapse Spark Pools**: Similar to Databricks, Synapse can use Spark pools to process streams and move data into a data warehouse for long-term storage and analysis. * **Integration with Azure Stream Analytics**: Synapse can also integrate with other services to perform complex event processing and real-time analysis on the data stream.

Security and Access Control: Shared Access Policies (SAS)

To ensure that only authorized producers and consumers can interact with an Event Hub, Azure uses **Shared Access Policies**. These policies define the specific permissions that a user or application has.

There are three primary permissions that can be granted in an Event Hub policy:

1. **Send**: This permission is granted to **producers**. It allows them to send events to the Event Hub. Without this permission, a producer cannot share its information or events.
2. **Listen**: This permission is granted to **consumers**. It allows them to read and receive events from the Event Hub.
3. **Manage**: This is the highest level of permission, allowing a user to manage the Event Hub itself, including its settings, security policies, and other configurations.

By correctly assigning these policies, you can ensure that each component of your data system has exactly the access it needs, keeping your data secure and your pipeline running smoothly.

WebApp to EventHubs

Tuesday, 3 March 2026 9:54 PM

Azure Event Hubs: Sending Events with Python

Introduction

This guide outlines the steps required to send events from a producer application to Azure Event Hubs using Python. It covers the necessary library installation, retrieval of the connection string from the Azure portal, and a Python code template for event submission.

Prerequisites

Before proceeding, ensure you have: * An active Azure subscription. * An Azure Event Hubs Namespace and an Event Hub created within it. * Python 3.7+ installed.

Step 1: Install the Azure Event Hubs Python SDK

To interact with Azure Event Hubs from Python, you need to install the official Azure Event Hubs client library. This can be done using `pip`:

```
pip install azure-eventhub
```

Step 2: Retrieve the Event Hubs Connection String

The connection string provides the necessary credentials and endpoint information for your application to connect to the Event Hub. Follow these steps to obtain it from the Azure portal:

1. Navigate to your **Event Hub Namespace** in the Azure portal.
2. In the left-hand menu, under **Settings**, select **Shared access policies**.
3. On the **Shared access policies** page, select the policy named **RootManageSharedAccessKey**. This policy typically has `Manage`, `Send`, and `Listen` permissions, making it suitable for both producers and consumers.
4. In the **Policy: RootManageSharedAccessKey** window, locate the **Primary Connection String** field. Click the copy button next to it to copy the connection string to your clipboard. It is recommended to paste this value into a secure temporary location (e.g., Notepad) for later use.

Step 3: Implement the Python Producer Code

Below is a Python code template that demonstrates how to create an Event Hub producer client and send a batch of events. Replace the placeholder values for `EVENT_HUB_CONNECTION_STR` and `EVENT_HUB_NAME` with your actual connection string and Event Hub name.

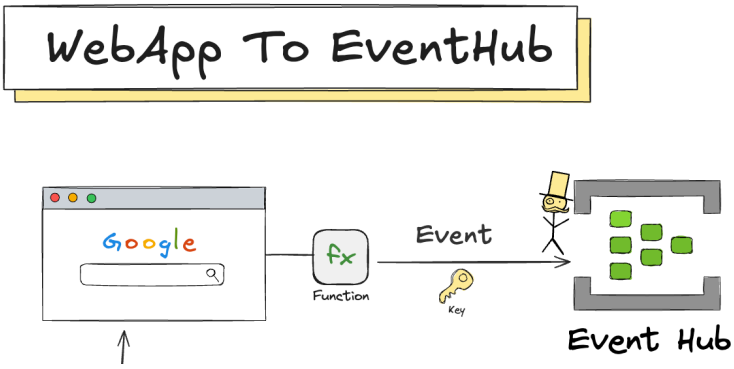
```
import asyncio


from azure.eventhub import EventData

from azure.eventhub.aio import EventHubProducerClient


EVENT_HUB_CONNECTION_STR = "YOUR_EVENT_HUB_CONNECTION_STRING"

EVENT_HUB_NAME = "YOUR_EVENT_HUB_NAME"


async def run():

    # Create a producer client to send messages to the event hub
```

```
    # Create a producer client to send messages to the event hub.
    # Specify a connection string to your event hubs namespace and
    # the event hub name.
    producer = EventHubProducerClient.from_connection_string(
        conn_str=EVENT_HUB_CONNECTION_STR, eventhub_name=EVENT_HUB_NAME
    )
    async with producer:
        # Create a batch.
        event_data_batch = await producer.create_batch()

        # Add events to the batch.
        event_data_batch.add(EventData("First event "))
        event_data_batch.add(EventData("Second event"))
        event_data_batch.add(EventData("Third event"))

        # Send the batch of events to the event hub.
        await producer.send_batch(event_data_batch)

asyncio.run(run())
```

Explanation of the Code

- **`EventHubProducerClient.from_connection_string`**: This method initializes the producer client using your Event Hubs connection string and the specific Event Hub name.
- **`producer.create_batch()`**: Events are typically sent in batches for efficiency. This method creates an `EventDataBatch` object.
- **`event_data_batch.add(EventData("..."))`**: Individual events are created using `EventData` and added to the batch.
- **`producer.send_batch(event_data_batch)`**: This asynchronously sends the entire batch of events to the Event Hub.
- **`asyncio.run(run())`**: The `asyncio` library is used to run the asynchronous `run()` function, which handles the event sending process.

By following these steps, you can successfully configure a Python application to act as a producer and send events to your Azure Event Hub.



Book

Setting-Up Codes and Directories

Wednesday, 4 March 2026 11:52 PM

Uber Data Engineering Project Setup Guide

Introduction

This guide provides a step-by-step process to set up and run the Uber Data Engineering Project. It covers environment preparation, repository cloning, dependency installation, and execution of the core scripts for connecting to Event Hubs and running the local web application.

Prerequisites

- A Linux-based environment (e.g., Ubuntu).
- `git` installed for cloning repositories.
- Python 3.x installed.
- An Azure Event Hubs instance configured with appropriate connection strings (as detailed in previous documentation).

Step 1: Install uv Package Manager

uv is an extremely fast Python package and project manager. While `brew install uv` is commonly used on macOS, for a Linux environment like Ubuntu, it's recommended to use the standalone installer via `curl`.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

After installation, ensure uv is added to your system's PATH. The installer typically provides instructions for this.

Step 2: Create Project Directory

Create a new directory for your project. This will be the main workspace for your Uber Data Engineering project.

```
mkdir "uber projects"
```

```
cd "uber projects"
```

Step 3: Clone the Git Repository

Navigate into your newly created project directory and clone the Uber Data Engineer Project repository. It's important to clone the repository directly rather than using `uv init` with the repository URL, as `uv init` is designed for initializing a new project with a specific structure, not for cloning an existing one.

```
git clone https://github.com/anshlambagit/Uber_Data_Engineer_Project.git
```

This will create a subdirectory named `Uber_Data_Engineer_Project` within your `uber projects` folder.

Step 4: Install Dependencies

Navigate into the cloned repository directory and install the required Python dependencies. It's best practice to do this within a virtual environment to isolate project dependencies.

```
cd Uber_Data_Engineer_Project
```

```
# Create a virtual environment using uv

uv venv
```

```
# Activate the virtual environment

source .venv/bin/activate


# Install dependencies from requirements.txt using uv pip

uv pip install -r requirements.txt
```

Step 5: Configure Event Hubs Connection (connection.py)

Before running the web application, you need to ensure your Event Hubs connection is correctly configured. The `connection.py` script (or similar logic within your project) will handle establishing the connection to Azure Event Hubs. You will likely need to update placeholder connection strings and Event Hub names within this file or in environment variables, as per previous documentation.

To test the connection:

```
python connection.py
```

Step 6: Run the Web Application (api.py)

Once the Event Hubs connection is verified, you can run the local web application. This application will act as a producer, sending data (events) to your configured Azure Event Hub.

```
python api.py
```

After running `api.py`, the web application should be accessible locally (e.g., via `http://localhost:8000` or similar, depending on the application's configuration). You can then interact with the web application to generate events and verify that they are being sent to your Event Hubs instance. You would typically monitor your Event Hubs metrics or have a consumer application running to confirm data ingestion.

ADF-DATALAKE

Saturday, 7 March 2026 7:09 PM

Azure Data Factory (ADF) Ingestion for Uber Project

Introduction

This document outlines the process of setting up an Azure Data Factory (ADF) pipeline to ingest data from a GitHub repository into Azure Data Lake Storage Gen2 (ADLS Gen2). This approach leverages ADF's low-code/no-code capabilities for efficient data pulling, migration, and orchestration, forming the initial **Bronze layer** of a data lake architecture.

1. Overview of the Uber Project Data Source

In this Uber project, a GitHub repository serves as the primary source for storing long-term and static data. While various data sources are available in real-world scenarios, GitHub is utilized here to demonstrate data ingestion from an external web-based source.

2. Azure Data Factory (ADF) Role

Azure Data Factory (ADF) is a cloud-based ETL/ELT service designed for data integration. It excels in: * **Data Pulling and Migration**: Efficiently extracting data from diverse sources. * **Data Orchestration**: Managing and scheduling data movement and transformation workflows. * **Data Movement**: Transferring data between various data stores.

ADF is a low-code/no-code resource, making it highly effective for initial data migration tasks. For heavy data transformations, it is generally recommended to extract and load data directly into ADLS Gen2 (the Bronze layer) and perform complex transformations within the data lake using other tools like Azure Databricks or Apache Spark.

3. Azure Data Lake Storage Gen2 (ADLS Gen2)

Azure Data Lake Storage Gen2 (ADLS Gen2) is chosen as the destination for ingested data due to its capability to store **hierarchical folders**. Unlike traditional Azure Blob Storage, ADLS Gen2 is optimized for big data analytics workloads and supports a file system semantic, which is crucial for organizing data effectively within a data lake.

4. Data Structure in the GitHub Repository

The GitHub repository contains a `Data` folder with important datasets: * **`bulk_rides.json`**: Represents initial, historical data (often referred to as an initial load or historical load). This data typically represents 5-10 years of historical information. * **Mapper/Mapping Data**: Other files in the `Data` folder are likely mapping datasets, used for lookup or reference purposes.

5. ADF Pipeline Creation: `HTTPStoADLF`

To load data from GitHub to ADLS Gen2 using ADF, a pipeline named `HTTPStoADLF` (or similar) is created. This pipeline will orchestrate the data migration.

5.1. Establishing Connections (Linked Services)

Before data migration, ADF needs to establish connections to both the source (GitHub) and the sink (ADLS Gen2).

- **GitHub Connection (HTTP Linked Service)**: A **Linked Service** is used to establish a connection between ADF and GitHub. This typically involves configuring an HTTP linked service, or a REST API linked service, to make API calls and fetch data from the GitHub repository.
- **Azure Data Lake Storage Gen2 Connection**: An ADLS Gen2 linked service is created to enable ADF to push the fetched data into the data lake. This requires a pre-existing **storage account** in the same resource group as the ADF instance.

5.2. Creating Datasets

Datasets in ADF represent the structure of the data within the linked services. For this project:

- **HTTP Dataset (Source)**: An HTTP dataset with JSON format is created to point to the GitHub URL (e.g., `https://raw.githubusercontent.com/`) where the data resides. This dataset helps ADF understand how to find the data within the large GitHub space.
- **ADLS Gen2 Dataset (Sink)**: An ADLS Gen2 dataset is created to define where the data will be stored in the data lake.

5.3. Metadata-Driven Ingestion for Multiple Datasets

Instead of creating a separate activity for each of the multiple datasets (e.g., 5-7 datasets), a **metadata-driven ingestion** approach is used. This involves creating a single pipeline that can handle multiple sources dynamically.

1. **Configuration Array**: A configuration file, typically a JSON array, is used to define the list of files to be ingested. This array can be stored directly within the pipeline parameters or as a file in a container within ADLS Gen2 (e.g., `files_array.json` in a container named `raw`).

```
[
  {"file":"map_cities"},
  {"file":"map_cancellation_reasons"},
  {"file":"map_rides"},
  {"file":"map_payment_methods"},
  {"file":"map_ride_statuses"},
  {"file":"map_vehicle_makes"},
  {"file":"map_types"}
]
```

2. **Lookup Activity**: A **Lookup activity** is added to the ADF pipeline. This activity reads the `files_array.json` from the ADLS Gen2 container (e.g., `raw` container), retrieving the list of files to be processed.

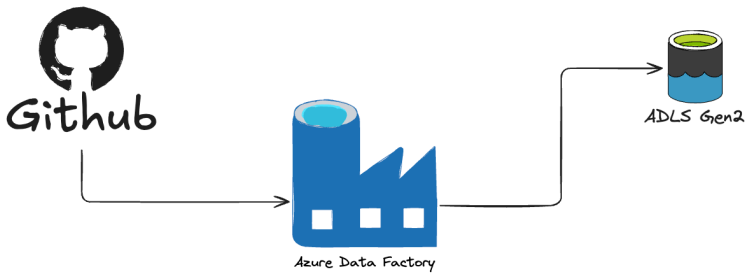
3. **ForEach Activity**: A **ForEach activity** is then used to iterate through the output of the Lookup activity (the `files_array`). This creates a loop that processes each file defined in the array. This is an example of **stateful orchestration**, where the output of a previous activity (Lookup) is used as input for a subsequent activity (ForEach).

4. **Copy Data Activity**: The core **Copy Data activity** is placed inside the ForEach loop. This activity performs the actual data migration for each file.

   - **Source Configuration**: The source dataset (`ds_github`) is configured with a dynamic path to fetch each file from the GitHub repository. This is achieved using a parameter, for example: `@{dataset().p_file}` which dynamically resolves to the file name from the `files_array`.
   - **Sink Configuration**: The sink dataset (`ds_ingest`) is configured with a dynamic file path to ensure that each file is stored separately in ADLS Gen2. A parameter like `@{dataset.p_file()}.json` can be used to dynamically name the output JSON files.

6. Debugging and Verification

After configuring the pipeline, it is crucial to debug and test it thoroughly to ensure that data is fetched correctly and stored in the designated location within ADLS Gen2. Upon successful execution, all files from the GitHub source will be fetched and stored in a specified folder (e.g., `ingestion` folder within the `raw` container) in the data lake. This completes the ADF pipeline for initial data ingestion.


Github
Azure Data Factory
ADLS Gen2

EventHub to DataBricks

Saturday, 14 March 2026 7:03 PM

Uber Project: Real-Time Data Ingestion with Spark Streaming and DLT

1. Overview

The ingestion phase involves pulling real-time events from Azure Event Hubs into Azure Databricks. This process populates the **Bronze layer** of the data lake, serving as the raw landing zone for all streaming ride data.

2. Environment Setup in Azure Databricks

The ingestion process is managed within an Azure Databricks workspace.

- **Catalog**: A catalog (e.g., `uber`) acts as the top-level container for data. It follows a three-level namespace: `catalog.schema.table`.
- **Schemas**: Within the catalog, a new schema named `bronze` is created to organize the raw ingestion tables.
- **Volumes**: A **Volume** (e.g., `my_volume`) is created within the schema to govern file storage, such as checkpoint directories.

3. Spark Declarative Pipelines (SDP) and Delta Live Tables

This project utilizes **Spark Declarative Pipelines (SDP)**, a modern framework implemented through **Delta Live Tables (DLT)**.

Advantages of SDP/DLT:

- **Declarative Processing**: Focus on *what* data to process rather than *how* to implement the low-level logic.
- **Automatic Orchestration**: Databricks handles the DAG, dependency management, and runtime optimization automatically.
- **Unified Batch and Stream**: Supports both batch and streaming workloads using SQL or Python.
- **Incremental Processing**: Efficiently processes only new data as it arrives.

4. Project Structure

The project is organized into specific folders to separate concerns: * **Exploration**: Used for testing PySpark code; not included in production. * **Transformation**: Contains the production-ready DLT pipeline code (DAG). * **Utilities**: Stores complex logic builders and helper functions for the pipelines.

5. Implementation: `uber_rides_ingestion`

The following code defines the streaming ingestion from Azure Event Hubs into the `rides_raw` table.

5.1. Imports and Configuration

```
from pyspark import pipelines as dp

from pyspark.sql.functions import *

from pyspark.sql.types import *


# Event Hubs configuration

EH_NAMESPACE = "ubereventshub"

EH_NAME = "ubertopic"
```

```
# Securely retrieve the connection string
EH_CONN_STR = spark.conf.get("connection_string")


KAFKA_OPTIONS = {
  "kafka.bootstrap.servers"  : f"{EH_NAMESPACE}.servicebus.windows.net:9093",
  "subscribe"                : EH_NAME,
  "kafka.sasl.mechanism"     : "PLAIN",
  "kafka.security.protocol"  : "SASL_SSL",
  "kafka.sasl.jaas.config"   :
f"kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule required username=
\"$ConnectionString\" password=\"{EH_CONN_STR}\";",
  "kafka.request.timeout.ms" : 10000,
  "kafka.session.timeout.ms" : 10000,
  "maxOffsetsPerTrigger"     : 10000,
  "failOnDataLoss"           : 'true',
  "startingOffsets"          : 'earliest'
}
```

5.2. Defining the Streaming Table

```
@dp.table
def rides_raw():
    # 1. Connect and read from Event Hub (Kafka endpoint)
    df = spark.readStream.format("kafka")\
                .options(**KAFKA_OPTIONS)\
                .load()


    # 2. Convert binary 'value' column to string
    # Kafka format: key | value | timestamp | topic
    df = df.withColumn("rides", col("value").cast("string"))


    return df
```

6. Testing and Checkpointing

Before moving to production, the logic is tested in the **Exploration** folder.

Checkpointing

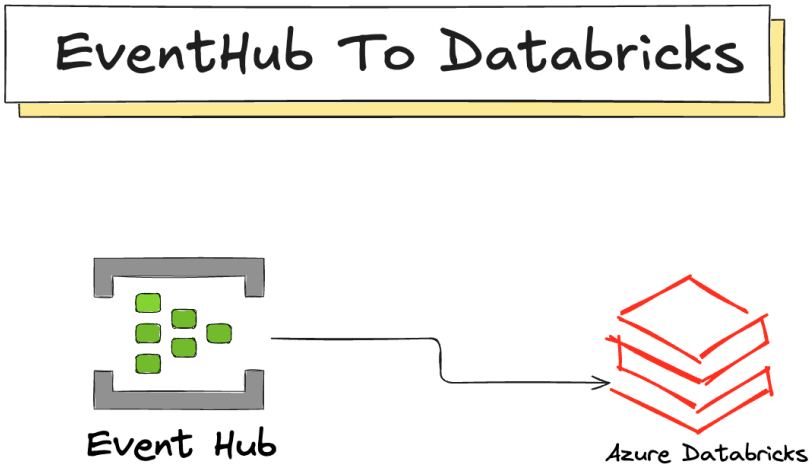
To ensure fault tolerance, a checkpoint location is defined. This records the stream's progress, allowing it to resume from the last successful offset after a failure.

```
display(df, checkpointLocation = "/Volumes/uber/bronze/my_volume/volume_folder/")
```

7. Production Execution

7. Production Execution

When the DLT pipeline is run, the `rides_raw` table is automatically created and updated in the `uber.bronze` schema. Every new event added to the `ubertopic` in Event Hubs is automatically pushed and ingested into the `rides_raw` table, providing a continuous real-time data stream for the Bronze layer.



ADLS to DataBricks

Sunday, 15 March 2026 1:10 AM

Azure Data Lake Storage (ADLS) Access and Bronze Layer Table Creation

Introduction

This document details the process of accessing data stored in Azure Data Lake Storage Gen2 (ADLS Gen2) from Azure Databricks, focusing on scenarios where direct access connectors are not available or preferred. It covers the use of Shared Access Signatures (SAS) tokens for secure data retrieval, dynamic ingestion of static mapping files into the Bronze layer, and the foundational concept of a One Big Table (OBT).

1. Accessing ADLS Gen2 Data from Azure Databricks

Azure Databricks and ADLS Gen2 are distinct Azure resources. To enable Databricks to access data residing in ADLS Gen2, a secure connection mechanism is required.

1.1. Access Connectors (Premium Edition)

Databricks offers **Access Connectors** as a secure and integrated way to connect to ADLS Gen2. However, this feature is typically available in Databricks premium editions. For standard or free editions, alternative methods are necessary.

1.2. Using Shared Access Signatures (SAS) Tokens

A common and flexible method to access ADLS Gen2 data from any application, including Databricks, is by using **Shared Access Signatures (SAS) tokens**. A SAS token is a URI that grants restricted access rights to Azure Storage resources.

- **Creation**: SAS tokens can be generated at either the **container level** or **folder level** within your ADLS Gen2 storage account. This allows for fine-grained control over which data can be accessed.
- **Purpose**: For this project, SAS tokens are used to securely read mapping files stored in the `raw` container of ADLS Gen2.

Example: Accessing a Single JSON File with SAS Token

The following Python code snippet demonstrates how to access a JSON file from ADLS Gen2 using its public URL appended with a SAS token:

```
import pandas as pd


url = "https://dluberprojectstorage.blob.core.windows.net/raw/ingestion/map_cities.json?sp=r&st=2026-03-14T19:35:48Z&se=2026-03-17T11:50:48Z&sr=c&sig=eJTMSUEmT8D8dbT8WEO6haXbf5ZPdsAmPO0AunVKNwE%3D"

df = pd.read_json(url)

df.head()
```

This approach allows Databricks (or any other client with the SAS token) to read the data directly.

2. Ingesting Static Mapping Files into the Bronze Layer

For static, infrequently changing data such as mapping files, a different ingestion strategy is employed compared to real-time streaming data. While real-time data often uses PySpark streaming libraries, static files can be efficiently processed using Pandas for initial loading and then converted to Spark DataFrames for storage in Delta format.

2.1. Dynamic Loading of Multiple Mapping Files

To load multiple mapping files dynamically, a loop iterates through a predefined list of files. Each file is read using Pandas, converted to a Spark DataFrame, and then saved as a Delta table in the Bronze layer.

```
import pandas as pd


files = [
    {"file": "map_cities"},
    {"file":"map_cancellation_reasons"},
    {"file":"map_payment_methods"},
    {"file":"map_ride_statuses"},
    {"file":"map_vehicle_makes"},
    {"file":"map_vehicle_types"}
]


for file in files:
    # Construct the URL with the SAS token (ensure SAS token is appended securely)
    # Note: The SAS token should be dynamically added or managed securely, not hardcoded.
    url =
f"https://dluberprojectdev.blob.core.windows.net/raw/ingestion/{file["file"]}.json?"


    # Read JSON data using Pandas
    df = pd.read_json(url)


    # Convert Pandas DataFrame to Spark DataFrame
    df_spark = spark.createDataFrame(df)


    # Writing Data To the Bronze Layer as Delta tables
    df_spark.write.format("delta")\
              .mode("overwrite")\
              .option("overwriteSchema", "true")\
              .saveAsTable(f"uber.bronze.{file["file"]}")
```

This code dynamically creates a Delta table for each mapping file (e.g., `uber.bronze.map_cities`, `uber.bronze.map_cancellation_reasons`, etc.) within the Databricks catalog.

2.2. Verifying Table Creation

After running the ingestion process, you can verify the successful creation and migration of tables using SQL commands within Databricks:

```
SELECT * FROM uber.bronze.map_cities
```

This command allows you to inspect the data within the newly created Delta table.

3. Bronze Layer Overview

At this stage, the Bronze layer is fully populated with: * **Real-time streaming data**: Ingested from Event Hubs (e.g., `rides_raw` table). * **Initial load data**: Historical data like `bulk_rides.json` (though its processing into a combined table is a Silver layer task). * **Static mapping tables**: Dynamically loaded from ADLS Gen2.

4. Transition to Silver Layer and One Big Table (OBT)

Combining the initial load data with the streaming data, and further processing to create a unified view, is typically a responsibility of the **Silver layer**. This often leads to the concept of a **One Big Table (OBT)**.

One Big Table (OBT)

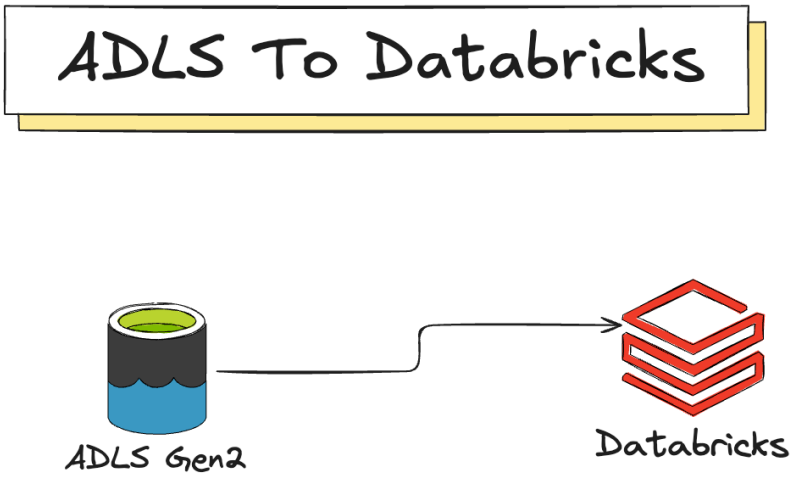
An OBT is a denormalized table that consolidates data from various sources and dimensions into a single, comprehensive table. Its advantages include: * **Simplified Data Model**: Reduces the complexity of managing separate data models for different domains. * **Streamlined Consumption**: Provides a single source of truth that various downstream domains can consume and then further refine based on their specific requirements, rather than receiving specialized data directly.

5. Data Format Preference: Delta over JSON

Throughout this process, the **Delta format** is preferred over raw JSON files for storing data in the data lake. Delta Lake offers significant advantages, including ACID transactions, schema enforcement, schema evolution, and improved performance for large-scale data operations, which are not inherent in plain JSON files.

6. End-to-End Flow Verification

To ensure the entire data pipeline is functioning correctly, it is recommended to run the `api.py` (the web application producer) to generate new records and observe if they are successfully ingested through the Event Hubs, Spark Streaming pipeline, and ultimately reflected in the Bronze layer tables.



Silver Layer (Transformation)

Sunday, 15 March 2026 11:36 AM

Uber Project: Silver Layer Transformation and One Big Table (OBT)

1. Introduction to the Silver Layer

The primary purpose of the **Silver layer** is **transformation**. It acts as a staging area where raw data from the Bronze layer is cleaned, structured, and unified. In this project, the Silver layer focuses on merging historical data (initial load) with real-time streaming data (incremental load) to create a comprehensive, queryable dataset.

2. The Challenge of Unifying Historical and Real-Time Data

In the Bronze layer, we have two distinct sources: * **`bulk_rides`**: Historical data containing potentially decades of ride records. * **`rides_raw`**: Real-time streaming data ingested from Azure Event Hubs.

Why a Simple Union is Inefficient

Performing a standard `UNION` between these two tables in every iteration (e.g., every 5 minutes) is not feasible. As the historical dataset grows into "big data," re-processing it alongside every new stream batch becomes a significant performance bottleneck.

The Solution: Streaming Append Flow

The strategy is to create a new **streaming table** where: 1. The **historical data** (`bulk_rides`) is appended once during the initial pipeline run. 2. The **streaming data** (`rides_raw`) is continuously appended as new events arrive. 3. This unified table then serves as the foundation for joining with mapping tables to create the **One Big Table (OBT)**.

3. Implementation: Building the `stg_rides` Table

This process is implemented using **Delta Live Tables (DLT)** and the `append_flow` decorator, which allows multiple data sources to write into a single target table.

3.1. Defining the Schema

To convert the raw JSON strings from the stream into a structured format, a explicit schema (`StructType`) is defined. This schema mirrors the structure of the historical `bulk_rides` table.

```
from pyspark.sql.types import *


rides_schema = StructType([
    StructField('ride_id', StringType(), True),
    StructField('confirmation_number', StringType(), True),
    StructField('passenger_id', StringType(), True),
    StructField('driver_id', StringType(), True),
    StructField('vehicle_id', StringType(), True),
    # ... (additional fields like passenger_name, driver_rating, total_fare, etc.)
    StructField('booking_timestamp', TimestampType(), True),
    StructField('total_fare', DoubleType(), True)
```

```
])
```

3.2. Creating the Staging Table

A target streaming table named `stg_rides` is created to store the structured and unified data.

```
dp.create_streaming_table("stg_rides")
```

3.3. Appending Historical Data (Bulk Flow)

The historical data is read as a stream from the `bulk_rides` table. A type cast is performed on the `booking_timestamp` to ensure it is stored as a `timestamp` rather than a `string`, enabling time-based analytics and window functions.

```
@dp.append_flow(target = "stg_rides")

def rides_bulk():

    df = spark.readStream.table("bulk_rides")

    df = df.withColumn("booking_timestamp", col("booking_timestamp").cast("timestamp"))

    return df
```

3.4. Appending Real-Time Data (Streaming Flow)

The real-time data is read from `rides_raw`. Since the raw data is in JSON string format, the `from_json` function is used along with the `rides_schema` to parse the JSON into a structured format.

```
@dp.append_flow(target = "stg_rides")

def rides_stream():

    df = spark.readStream.table("rides_raw")

    # Parse JSON string to structured format and expand columns

    df_parsed = df.withColumn("parsed_rides", from_json(col("rides"), rides_schema))\

                .select("parsed_rides.*")

    return df_parsed
```

4. Data Normalization and Cleaning

Raw events often arrive in unstructured formats. For example, a single `rides` column might contain a complex JSON string: `{"ride_id": "10812...", "passenger_name": "Willie Powell", ...}`

The use of `from_json()` and `.select("parsed_rides.*")` effectively "flattens" this JSON string into individual, typed columns. This normalization ensures that the data in `stg_rides` is clean, consistent, and ready for analytical queries.

5. Verification and Incremental Processing

Once the pipeline is executed: * **Initial Run**: The `stg_rides` table will contain the union of historical records (e.g., 2,000+ records) and any existing stream data. * **Subsequent Runs**: Because DLT uses **incremental processing**, running the pipeline again when no new data has arrived will result in 0 new records being added. * **New Data**: If a new record is added via the `api.py` producer, only that specific new record will be processed and appended to `stg_rides`.

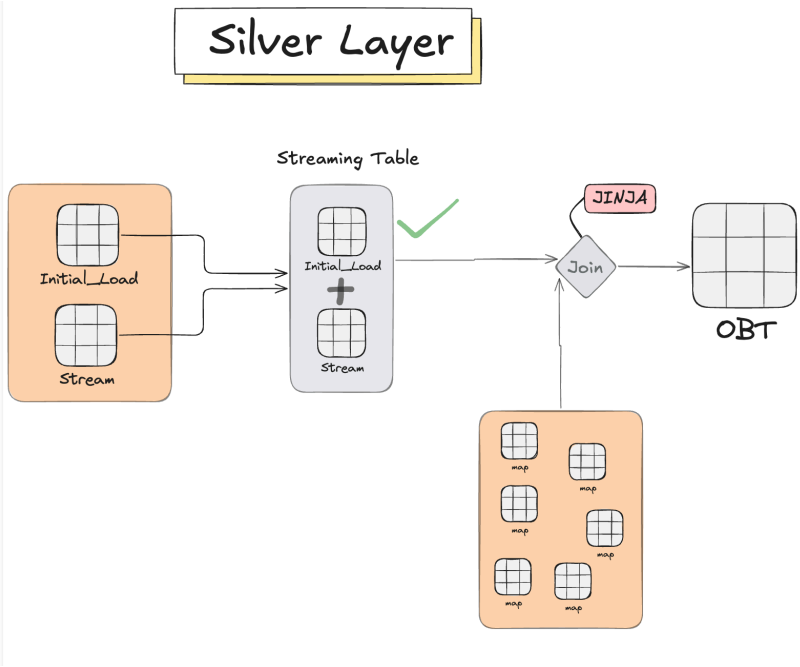
6. Architecture Evolution

The data pipeline architecture has now evolved to:

```
Uber Ride App → Azure Event Hub → rides_raw (Bronze) → stg_rides (Silver)
```

The `stg_rides` table in the Silver layer now serves as the high-quality, structured staging area for the

final transition to the Gold layer.



Metadriven Pipeline for Silver Layer

Sunday, 15 March 2026 7:02 PM

Uber Project: Metadata-Driven Silver Layer with Jinja Templating

1. Introduction to Metadata-Driven Pipelines

As data ecosystems grow, manually maintaining complex SQL queries for joining numerous tables becomes unsustainable. In this project, we implement a **metadata-driven pipeline** for the Silver layer using **Jinja templating**. This approach allows us to define the pipeline's structure (tables, joins, and columns) in a configuration file, which is then used to dynamically generate the required SQL.

2. The Power of Jinja Templating

Jinja is a powerful template engine for Python. By treating SQL as text, we can use Jinja to inject logic—such as loops and conditional statements—directly into our SQL queries.

Why Use Jinja?

- **Scalability**: Adding a new table or joining key only requires updating a simple configuration file, rather than rewriting long, complex SQL scripts.
- **Maintainability**: It significantly reduces the risk of breaking code during updates, as the core SQL generation logic remains consistent.
- **Dynamic Generation**: The pipeline automatically picks up table names, joining conditions, and transformations from the metadata.

3. Implementation: The SQL Generator

The following Jinja template is designed to iterate through a configuration list (`jinja_config`) and build a comprehensive `SELECT` statement with multiple `LEFT JOIN`s.

3.1. The Jinja Template

```
from jinja2 import Template


jinja_str = """
    SELECT
        {% for config in jinja_config %}
            {{ config.select }}
                {% if not loop.last %}
                    ,
                {% endif %}
        {% endfor %}
    FROM
        {% for config in jinja_config %}
            {% if loop.first %}
                {{ config.table }}
            {% else %}
                LEFT JOIN {{ config.table }} ON {{ config.on }}
```

```
            {% endif %}
        {% endfor %}


        {% for  config in jinja_config %}
            {% if loop.first %}
               {% if config.where != "" %}
                WHERE
                {% endif %}
            {% endif %}
            {{ config.where }}
                {% if not loop.last %}
                  {% if config.where != "" %}
                    AND
                  {% endif %}
                {% endif %}
        {% endfor %}
"""
```

3.2. Configuration Metadata (`jinja_config`)

The `jinja_config` serves as the blueprint for the One Big Table (OBT). It defines the base table (`stg_rides`) and all necessary lookup tables (mappings) to convert IDs into human-readable information.

Table Alias	Source Table	Joining Condition	Purpose
`stg_rides`	`uber.bronze.stg_rides`	(Base Table)	Contains core ride data and IDs.
`map_vehicle_makes`	`uber.bronze.map_vehicle_makes`	`stg_rides.vehicle_make_id = map_vehicle_makes.vehicle_make_id`	Converts make ID to name (e.g., Toyota).
`map_vehicle_types`	`uber.bronze.map_vehicle_types`	`stg_rides.vehicle_type_id = map_vehicle_types.vehicle_type_id`	Adds type (SUV, Sedan) and rates.
`map_ride_statuses`	`uber.bronze.map_ride_statuses`	`stg_rides.ride_status_id = map_ride_statuses.ride_status_id`	Converts status ID (e.g., Completed).
`map_cities`	`uber.bronze.map_cities`	`stg_rides.pickup_city_id = map_cities.city_id`	Adds city, state, and region info.

4. Building the One Big Table (OBT)

The primary goal of this metadata-driven approach is to create an **enriched rides table**, often called a **One Big Table (OBT)**. By joining the main `stg_rides` table with multiple mapping tables, we transform raw IDs into descriptive attributes.

- **Example**: `vehicle_type_id = 2` becomes `vehicle_type = SUV`.
- **Result**: A single, wide table that downstream teams can easily consume for their specific analytical needs.

5. Handling Data Changes: Slowly Changing Dimensions (SCD)

In real-world scenarios, historical data is rarely static. To handle updates to dimensions (like city information or vehicle rates), we implement **Slowly Changing Dimensions (SCD)**, specifically **SCD Type 2**

multiple mapping tables, we transform raw IDs into descriptive attributes.

- **Example**: `vehicle_type_id = 2` becomes `vehicle_type = SUV`.
- **Result**: A single, wide table that downstream teams can easily consume for their specific analytical needs.

5. Handling Data Changes: Slowly Changing Dimensions (SCD)

In real-world scenarios, historical data is rarely static. To handle updates to dimensions (like city information or vehicle rates), we implement **Slowly Changing Dimensions (SCD)**, specifically **SCD Type 2**.

Implementation Details:

- **Timestamps**: An `updated_at` field is added to mapping files (e.g., `map_cities`) to track when records change.
- **ADF & Bronze Updates**: The ADF pipeline is re-run to fetch updated JSON files, and the Bronze layer tables are overwritten using the `overwriteSchema` option to accommodate the new timestamp columns.
- **Silver Transformation**: The Silver layer pipeline is updated to ensure `booking_timestamp` and other date fields are correctly cast to `TimestampType()` for accurate time-based tracking.

6. Verification

The generated SQL can be tested and displayed in Databricks to verify the OBT structure:

```
template = Template(jinja_str)

rendered_template = template.render(jinja_config=jinja_config)

display(spark.sql(rendered_template))
```

This process ensures the Silver layer OBT is perfectly structured and ready for final consumption in the Gold layer.

Silver OBT Watermarking

Monday, 16 March 2026 10:43 AM

Uber Project: Streaming Joins, Watermarking, and the Silver OBT

1. Introduction to Streaming Joins in the Silver Layer

The final step in the Silver layer is the creation of the **Silver One Big Table (OBT)**. This involves joining the streaming staging table (`stg_rides`) with various batch mapping tables. To handle this in a real-time environment, we utilize **Streaming Joins** within the Spark Declarative Pipeline (SDP) framework.

2. Managing Late Data with Watermarking

A critical challenge in streaming architectures is handling events that arrive late due to network latency or other issues. To address this, we use the concept of **Watermarking**.

What is a Watermark?

- **Definition**: A watermark is a threshold that tells Spark how long to wait for late-arriving events.
- **State Management**: It is essential for **stateful transformations**. Since Spark cannot remember the state of all events indefinitely, a watermark sets a deadline.
- **Implementation**: In our pipeline, we set a watermark on the `booking_timestamp` with a delay of **5 minutes**.
  - **Example**: If a ride occurs at 10:00 AM but the event arrives at 10:04 AM, it is processed. If it arrives at 10:06 AM, it may be dropped to prevent infinite waiting and resource exhaustion.

3. Implementation: `silver_obt` Streaming Table

The `silver_obt` table is defined as a streaming table that continuously updates as new ride events arrive. It uses the `STREAM` keyword to indicate incremental reading from the source.

3.1. SQL Implementation with DLT

```
CREATE OR REFRESH STREAMING TABLE silver_obt

AS

    SELECT

        stg_rides.*, -- Core ride data

        map_vehicle_makes.vehicle_make,

        map_vehicle_types.vehicle_type, map_vehicle_types.description,
map_vehicle_types.base_rate,

        map_ride_statuses.ride_status,

        map_payment_methods.payment_method, map_payment_methods.is_card,

        map_cities.city as pickup_city, map_cities.state, map_cities.region,

        map_cancellation_reasons.cancellation_reason

    FROM

        STREAM(uber.bronze.stg_rides)

        WATERMARK booking_timestamp DELAY OF INTERVAL 5 MINUTES stg_rides

    LEFT JOIN uber.bronze.map_vehicle_makes map_vehicle_makes

        ON stg_rides.vehicle_make_id = map_vehicle_makes.vehicle_make_id

    LEFT JOIN uber.bronze.map_vehicle_types map_vehicle_types
```

```
    LEFT JOIN uber.bronze.map_vehicle_types map_vehicle_types
        ON stg_rides.vehicle_type_id = map_vehicle_types.vehicle_type_id
    LEFT JOIN uber.bronze.map_ride_statuses map_ride_statuses
        ON stg_rides.ride_status_id = map_ride_statuses.ride_status_id
    LEFT JOIN uber.bronze.map_payment_methods map_payment_methods
        ON stg_rides.payment_method_id = map_payment_methods.payment_method_id
    LEFT JOIN uber.bronze.map_cities map_cities
        ON stg_rides.pickup_city_id = map_cities.city_id
    LEFT JOIN uber.bronze.map_cancellation_reasons map_cancellation_reasons
        ON stg_rides.cancellation_reason_id =
map_cancellation_reasons.cancellation_reason_id
```

4. Key Components of the OBT Pipeline

- **`CREATE OR REFRESH STREAMING TABLE`**: Ensures the table is either created or updated incrementally with new data.
- **`STREAM(table_name)`**: Directs Spark to read the source table as a continuous stream rather than a static batch.
- **`LEFT JOIN` on Mapping Tables**: Converts technical IDs (e.g., `vehicle_type_id = 2`) into human-readable attributes (e.g., `vehicle_type = SUV`). This "flattens" the data into a wide, descriptive format.

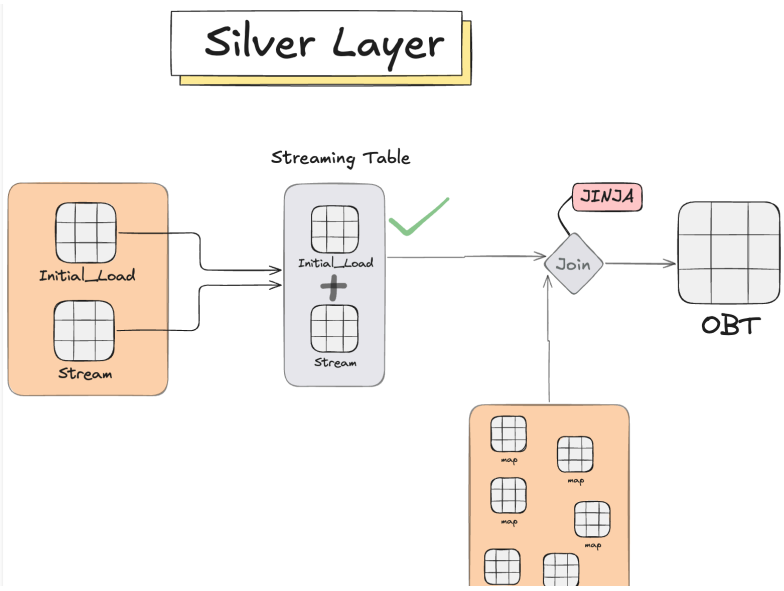
5. The One Big Table (OBT) Concept

The `silver_obt` is the definitive "One Big Table" for the project. By consolidating many small lookup tables into a single enriched dataset, we provide a high-quality source for: * **Business Intelligence (BI)**: Powering dashboards and reports. * **Analytics**: Enabling deep dives into ride patterns and performance. * **Machine Learning**: Providing clean, feature-rich data for model training.

6. Full Pipeline Summary

The end-to-end flow is now complete: 1. **Event Hub**: Receives raw events from the Ride App. 2. **`rides_raw` (Bronze)**: Stores raw JSON events. 3. **`stg_rides` (Silver)**: Stores structured, parsed, and unified (historical + streaming) data. 4. **`silver_obt` (Silver/Gold)**: The final enriched analytics table, joined with all necessary dimensions and protected by watermarking.

With the creation of the `silver_obt`, the Silver layer transformations are successfully completed.



Gold Layer Dimensional Modeling

Monday, 16 March 2026 5:39 PM

Uber Project: Gold Layer and Dimensional Data Modeling

1. Introduction to the Gold Layer

The **Gold layer** is the final and most critical stage of the data pipeline. Its primary objective is to deliver data models optimized for business consumption. In this project, we transition from the "One Big Table" (OBT) in the Silver layer to a **Dimensional Data Model** (Star Schema), consisting of **Fact** and **Dimension** tables.

2. Dimensional Modeling Strategy

Dimensional modeling organizes data into two main categories: * **Fact Tables**: Contain quantitative, numerical data (measures) that can be aggregated (e.g., total fare, distance, duration). * **Dimension Tables**: Contain contextual, descriptive data (attributes) that provide meaning to the facts (e.g., passenger details, driver information, location demographics).

Why Separate Facts and Dimensions?

- **Contextual Clarity**: Dimensions provide the "who, what, where, and when" of the business process.
- **Aggregation Efficiency**: Fact tables are optimized for high-speed numerical calculations.
- **Business Standards**: Standardizing dimensions (like IDs and dates) ensures consistency across the entire organization.

3. Implementing Change Data Capture (CDC) and SCD

To maintain accurate historical records, we use **Change Data Capture (CDC)** and **Slowly Changing Dimensions (SCD)**.

3.1. SCD Type 1: Overwrite (Dim Passenger, Driver, Vehicle, Payment)

For attributes where history is not required (e.g., updating a phone number), we use **SCD Type 1**. This method overwrites old values with new ones, ensuring the table always reflects the most current state.

3.2. SCD Type 2: Historical Tracking (Dim Location)

For attributes where tracking changes over time is critical (e.g., city names or regions updating), we use **SCD Type 2**. This method adds a new row for every change, logging the history with timestamps. * **Example**: If "Philadelphia" is updated to "New Philadelphia," SCD Type 2 creates a new record with the same primary key but a unique timestamp, allowing for "as-of" historical reporting.

4. Implementation: Gold Layer Pipeline

The Gold layer is built using **Delta Live Tables (DLT)** views and the `create_auto_cdc_flow` function to automate the ingestion from the Silver OBT into the final dimension tables.

4.1. Example: `dim_passenger` Implementation

```
from pyspark import pipelines as dp


@dp.view

def dim_passenger_view():

    # Read from Silver OBT and select relevant columns

    df = spark.readStream.table("silver_obt")

    df = df.select("passenger_id", "passenger_name", "passenger_email",
```

```
    df = df.select("passenger_id", "passenger_name", "passenger_email",
"passenger_phone")

    # Deduplicate based on ID (a passenger may have multiple ride records)

    df = df.dropDuplicates(subset=['passenger_id'])

    return df


# Create the target streaming table and define the CDC flow

dp.create_streaming_table("dim_passenger")

dp.create_auto_cdc_flow(

  target = "dim_passenger",

  source = "dim_passenger_view",

  keys = ["passenger_id"],

  sequence_by = "passenger_id",

  stored_as_scd_type = 1, # SCD Type 1: Overwrite

)
```

4.2. Dimensional Tables Summary

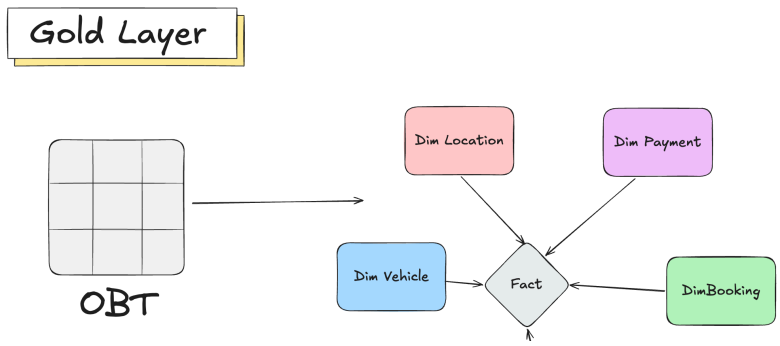
The following tables are created in the Gold layer:

Table Name	Type	Key	Purpose
`dim_passenger`	SCD Type 1	`passenger_id`	Stores unique passenger contact details.
`dim_driver`	SCD Type 1	`driver_id`	Stores driver credentials and ratings.
`dim_vehicle`	SCD Type 1	`vehicle_id`	Stores vehicle make, model, and color.
`dim_payment`	SCD Type 1	`payment_method_id`	Stores payment methods and auth requirements.
`dim_booking`	SCD Type 1	`ride_id`	Stores ride-specific IDs and timestamps.
`dim_location`	**SCD Type 2**	`pickup_city_id`	Tracks historical changes in city/region data.

5. Verification of the End-to-End Flow

To verify the Gold layer, new bookings can be generated via the `api.py` producer. The data flow can be observed as it progresses through the entire Medallion architecture: 1. **Bronze**: Raw events ingested into `rides_raw`. 2. **Silver**: Data parsed, unified, and enriched into `silver_obt`. 3. **Gold**: Enriched data is deduplicated and distributed into the final dimensional tables (`dim_*`), with CDC handling any updates or historical logging automatically.

With the completion of the Gold layer dimensional model, the Uber Data Engineering project is successfully finalized, providing a robust, scalable, and business-ready data warehouse.



Gold Layer Fact Table

Monday, 16 March 2026 11:11 PM

Uber Project: Gold Layer Fact Table and Data Warehouse Completion

1. Introduction to the Fact Table

The final component of the **Gold layer** is the **Fact table**. While dimension tables store descriptive context (the "who, what, where"), the Fact table stores the quantitative metrics (the "how much") and the foreign keys that link everything together. This table is the primary source for numerical analysis and business intelligence.

2. Implementation: Building the `fact` Table

The Fact table is created as a streaming table in the Gold layer, sourced from the enriched `silver_obt` table. It focuses on selecting only the numerical metrics and the IDs required to join with the dimension tables.

2.1. Defining the Fact View

A temporary streaming view is created to select the necessary columns and define the structure of the Fact table.

```
@dp.view
def fact_view():
    # Read from the enriched Silver OBT
    df = spark.readStream.table("uber.bronze.silver_obt")

    # Select foreign keys and numerical metrics
    df = df.select(
        "ride_id",              # Primary/Sequence Key
        "pickup_city_id",       # Foreign Key to dim_location
        "payment_method_id",    # Foreign Key to dim_payment
        "driver_id",            # Foreign Key to dim_driver
        "passenger_id",         # Foreign Key to dim_passenger
        "vehicle_id",           # Foreign Key to dim_vehicle
        "distance_miles",       # Metric
        "duration_minutes",     # Metric
        "base_fare",            # Metric
        "distance_fare",        # Metric
        "time_fare",            # Metric
        "surge_multiplier",     # Metric
        "total_fare",           # Metric
        "tip_amount",           # Metric
        "rating",               # Metric
```

```
        "base_rate",            # Metric
        "per_mile",             # Metric
        "per_minute"            # Metric
    )
    return df
```

2.2. Creating the Streaming Fact Table with CDC

The `fact` table is then created with an **Auto CDC Flow**. This ensures that any updates or new ride events are automatically handled.

```
dp.create_streaming_table("fact")
dp.create_auto_cdc_flow(
  target = "fact",
  source = "fact_view",
  keys = ["ride_id", "pickup_city_id", "payment_method_id", "driver_id", "passenger_id",
"vehicle_id"],
  sequence_by = "ride_id",
  stored_as_scd_type = 1, # Overwrite old values to keep current metrics
)
```

3. Final Data Warehouse Model (Star Schema)

With the Fact table and all Dimension tables in place, the project now follows a classic **Star Schema** architecture:

- **Center**: `fact` table (Ride metrics and IDs)
- **Surrounding Dimensions**:
  - `dim_passenger`
  - `dim_driver`
  - `dim_vehicle`
  - `dim_location` (SCD Type 2)
  - `dim_payment`
  - `dim_booking`

4. Querying the Data Warehouse

To perform analytics, you join the Fact table with the relevant Dimension tables. For SCD Type 2 tables (like `dim_location`), it is critical to filter for the **current active record** where the `__END_AT` timestamp is `NULL`.

Example: Revenue per Region

```
SELECT
    fact.ride_id,
    fact.base_fare,
    dim.region
FROM uber.bronze.fact AS fact
LEFT JOIN uber.bronze.dim_location AS dim
    ON fact.pickup_city_id = dim.pickup_city_id
    AND dim.`__END_AT` IS NULL -- Ensures we only get the latest version of the location
```

5. Project Conclusion and Data Ops

The Uber Data Engineering project is now fully operational, following the **Medallion Architecture**:

1. **Bronze (Raw)**: Ingesting raw JSON events from Event Hubs and ADLS Gen2.
2. **Silver (Staging/Enriched)**: Parsing, unifying historical and streaming data, and creating the One Big Table (OBT).
3. **Gold (Curated)**: Transforming the OBT into a Star Schema with automated CDC and SCD handling.

Next Steps: Scheduling

Once the pipeline is tested and verified, it can be scheduled to run at regular intervals (e.g., every 1-5 minutes) to provide near real-time business insights. This end-to-end automation falls under the domain of **Data Ops**, ensuring a robust, scalable, and self-healing data platform.